

Първо ще въведем две определения:

- оператор е единица на действие. В C++ поредица от оператори в {} е съставен оператор и сам по себе си е оператор
- израз е конструкция със стойност (константа, променлива, обръщение към функция)

Принципи на структурното програмиране

Структурното програмиране е парадигма, която няколко принципа за писане на програми. Основната цел е кодът да бъде по-разбираем, по-лесен за поправки и по-лесно проверим. Основните принципи са:

1. Сложна задача се разделя на краен брой по-прости задачи и всяка се решава самостоятелно
2. Всяка структурна единица (блок, функция, цикъл) има един вход и един изход
3. Използват се само три базови конструкции:
 - последователност - операторите се изпълняват един след друг
 - разклонение - избор между алтернативи в зависимост от условие
 - повторение - многократно изпълнение на даден блок докато е изпълнено условие

Тези три конструкции са достатъчни за всеки изчислим алгоритъм (Теорема на Блум-Якопин), което прави голямо излишество.

4. Всяка променлива се ограничава до възможно най-малък обхват на действие. Глобалните променливи се избягват, защото промените по тях се проследяват трудно
5. Повтарящи се действия се изнесат във функции, което намалява дублирането на код и риска от грешки при промяна

Управление на изчислителния процес

По поредиране операторите в програми се изпълняват един след друг.

Управляващите конструкции позволяват да променим това.

Условни оператори

Проверява се условие и в зависимост от това дали е true или false се изпълнява един или нито един от клоните.

Пример:

```
int x;  
std::cin >> x;  
if (x < 0) {  
    std::cout << „x е отрицателно“;  
} else if (x > 0) {  
    std::cout << „x е положително“;  
} else {  
    std::cout << „x е 0“;  
}
```

В горния пример се изпълнява един клон от трите издраски (стига x да е коректен тип), но ако имаме случая по-долу няма да се изпълни нито един от клоните, защото $x = 0$ и $x \neq 0$ и $x \neq 0$

```
int x = 0;  
if (x < 0) {  
    std::cout << „x е отрицателно“;  
} else if (x > 0) {  
    std::cout << „x е положително“;  
}
```

Има и терциарна операция, която за разлика от if връща стойност:

```
std::string str = (x < 0) ? „x е отрицателен“ : „x е неотрицателен“;
```

Има и switch оператор, който може да се използва така:

```
switch (израз) {  
    case 1: .....; break;  
    case 2: .....; break;  
    .....  
    default: .....;  
}
```

Важно е на края на всеки case да има break, защото иначе случаите под този който е съвпаднал с резултата от израз ще се изпълняват докато не се срещне break или не свърши switch оператора.

Оператори за цикли

Има три оператора за цикли в C++

- for операторът е подходящ когато знаем броя повторения например:

```
int n = 5;
for (int i = 0; i < n; ++i) {
    std::cout << i;
}
```

В този пример цикълът ще се изпълни 5 пъти. Инициализацията в скобите се изпълнява веднъж, условието се проверява преди всяко влизане, а след всяко излизане се изпълнява корекцията (тя може и да е по-сложна от ++i, например i=i+2)

- while цикълът е подходящ когато не знаем броя повторения, но искаме да се изпълнява докато някакво условие е истина. Например

```
int x;
std::cin >> x;
while (x >= 10) {
    x = x / 10;
}
```

- do/while е подходящ когато искаме поне 1 изпълнение дори условието да е false

```
int n;
do {
    std::cin >> n;
} while (n <= 0);
```

Трите оператора са базично изразими

• Оператори за прекъсване

`break` излиза от ний-вътрешния цикъл или `switch`

`continue` премества към следващата итерация на ний-вътрешния цикъл

На практика нарушават принципа за един вход - един изход, но все пак, за разлика от `goto`, водят към предвидимо място

Променливи

Всяка променлива има име, адрес, тип и стойност.

Дефиницията на променлива заделя памет за нея, а инициализацията задава стойност. Например:

```
int a; // неинициализирана, но е заделена памет
```

```
int b = 10; // инициализирано с 10
```

```
int c = a + b; // undefined behavior защото a не е инициализирана
```

Локалните променливи в C++ не се инициализират автоматично и зетенето от тях е `undefined behavior`

- Константите имат стойност, която не може да се променя след инициализиране. Те трябва да бъдат инициализирани при дефиниране

```
const int FINGERS = 10;
```

```
FINGERS = 12; // компилаторът няма да го допусне
```

- Локалните променливи се дефинират в тялото на функция или блок и съществуват до края на блока, в който са дефинирани и са видими само в него. Те се разполагат в стека
- Глобалните променливи се дефинират извън всички функции и съществуват през целия живот на програмата. Те не се разполагат в стека, а в специален сегмент (`data segment`)

```
int counter = 0;
```

```
void increment() {
```

```
    ++counter
```

```
}
```

```
void decrement() {
```

```
    --counter;
```

```
}
```

- Операторът за присвояване има следния синтаксис:

`<lvalue> = <rvalue>;`

- `lvalue` е обект който има определено място в паметта
- `rvalue` е временен обект, който няма определено място в паметта

Пример:

```
int x; int y;
```

`x = 5` // валидно `x` е `lvalue` и `5` е `rvalue`

`x + 1 = 6` // невалидно `x + 1` не е `lvalue`

`5 = x` // `5` не е `lvalue`

`x = y` // работи въпреки че `y` е `lvalue` в този случай стойността на `y` е `rvalue`

- В C++ функциите имат тип на резултат и/или параметри. Ако типът на резултата е `void`, то това е процедура, в противен случай е функция.

Пример

```
int square(int x) {
```

```
    return x * x;
```

```
}
```

```
void printHello() {
```

```
    std::cout << "Hello";
```

```
}
```

Възможно е функция да бъде само декларирана в сегашния файл и да бъде имплементирана в друг. Пример

functions.hpp

```
int square(int x);
```

functions.cpp

```
int square(int x) {
```

```
    return x * x;
```

```
}
```

• Параметри

- формални параметри са имената в счепатурата при дефиницията
- фактически параметри са изразите, които се подават при извикване

```
int square(int x) { // x е формален параметър
    return x*x;
}

int main() {
    int a=5;
    int b=square(a+1); // a+1 е фактически параметър
}
```

• Подаване по стойност

Прави се копие на стойността на фактическия параметър и променни във функцията засягат само копие.

```
void uselessIncrement(int x) {
    ++x;
}
```

```
int main() {
    int a=5;
    uselessIncrement(a);
    std::cout << a;
}
```

В горният пример в scope-а на uselessIncrement x има стойност 5, която става на 6, но това не засяга а в main функцията.

• Подаване по име

При подаването по име не се прави копие на фактическия параметър и променните в scope-а на функцията засягат фактическия параметър.

```
void increment(int& x) {
    ++x;
}

int main() {
    int a=5;
    increment(a);
    std::cout << a;
}
```

В този пример промяната засяга а от main. Тук е възможно да се каже че по име НЕ може да се прегледа value, защото няма определено място в паметта.

- Типове и проверка за съответствие на тип

Примитивни типове в C++

- bool - има стойност true/false
- char - символен тип 1 байт
- int (short, long, unsigned) - целочислен тип
- float, double, long double - числа с плаваща запетая

- Стандартни преобразувания

Преобразувания в които няма опасност за загуба на информация стават неявно, т.е. bool $\rightarrow$ char $\rightarrow$ short int $\rightarrow$ long int $\rightarrow$ float $\rightarrow$ double става неявно, но ако искаме да запишем double в int ще трябва експлицитно да кажем това, защото по този начин губим информация (числата след запетаята)

```
double a = 3.14;
```

```
int b = (int)a; // C стил
```

```
int c = static_cast<int>(a); // C++ стил
```